TECHNICAL IMPLEMENTATION REPORT

Support Ticket Tracker

Pure Truth Water Factory — Apex & Lightning Web Components

Africa Ohana — Developer Sprint — Final Project

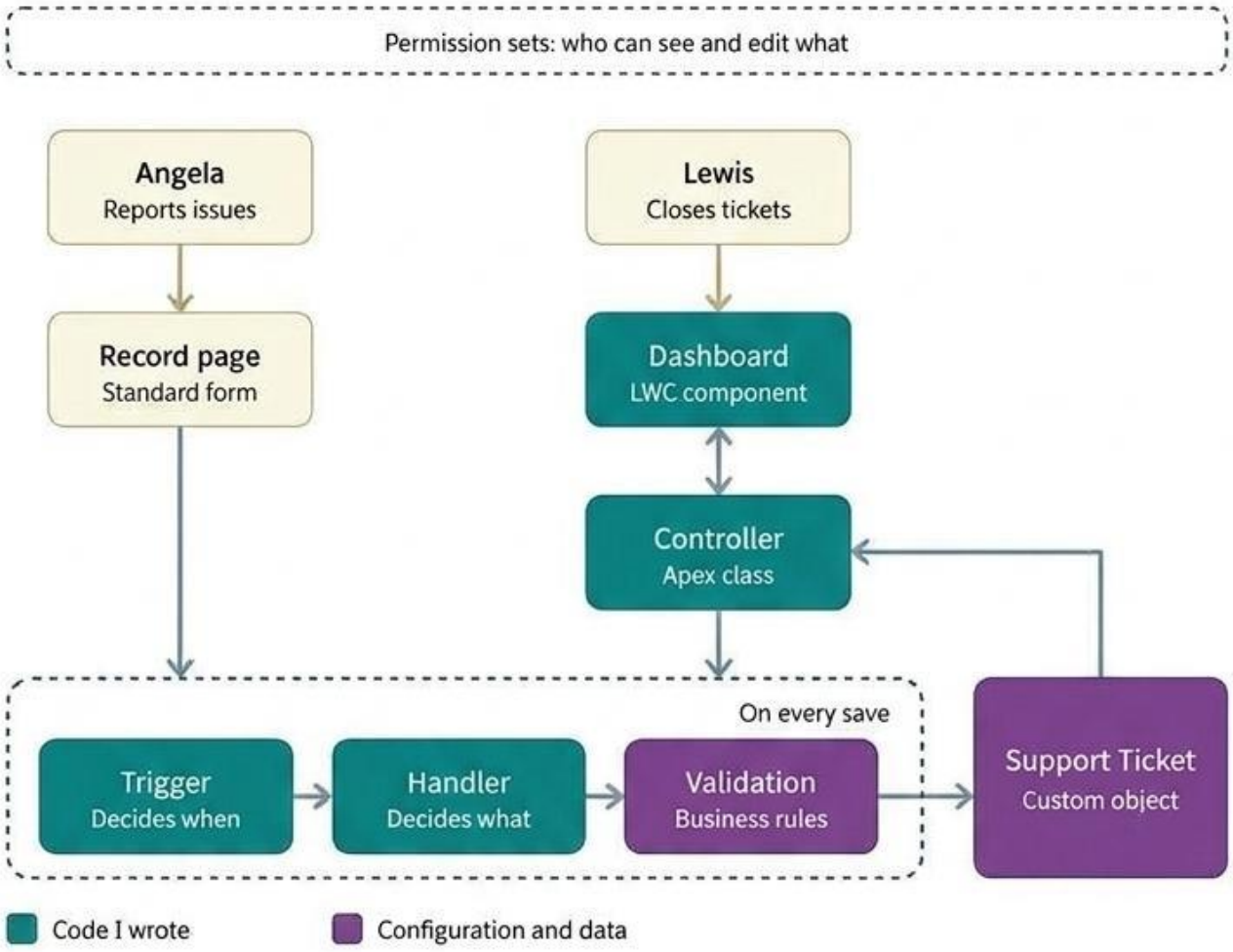
Author: Roberts, Marisa E.

Status: Final submission document

October 2026

Executive Summary

This report documents the design and implementation of the Support Ticket Tracker, a custom Salesforce solution built for Pure Truth Water Factory (PTWF) to replace informal, untracked incident reporting with a centralized and auditable workflow. The solution combines a custom object (Support_Ticket__c), a bulk-safe Apex Trigger and Handler pair, a secure Apex Controller, a reactive Lightning Web Component (ticketDashboard), two Permission Sets, and two Validation Rules. It is backed by a nine-method automated test suite, including 200-record bulk scenarios, with 92% overall code coverage (platform minimum: 75%).



1. Business Context

Pure Truth Water Factory is a water treatment, bottling, and distribution company operating on two main fronts:

- **Commercial / B2B:** bulk water distribution, together with leasing contracts for industrial machinery and dispensers, serving companies, gyms, and other industries.
- **Internal Operations and Supplies:** management of the delivery fleet, plant maintenance technicians, and the logistics, tools, and operational infrastructure that keep production running. The Support Ticket Tracker belongs to this second front.

1.1 Actors and Roles

Actor	Role	Responsibility in the solution
Angela James	Plant Supervisor (Standard User / Employee profile)	Detects and reports support and infrastructure incidents on the filtration plant and bottling lines. Primary ticket creator (Support_Ticket_Creator).
Lewis Webb	Finance Officer (Support Manager)	Responsible for operational continuity and resource control. Uses the centralized LWC panel to review incidents in priority order, coordinate resolution, and close tickets (Support_Ticket_Manager).

2. Problem Justification

Incident reporting at the plant currently happens through informal channels (WhatsApp, loose emails, hallway conversations). This causes three key issues:

- **Lack of visibility:** there is no unified list of pending claims.
- **No urgency distinction:** critical incidents are handled at the same pace as minor requests.
- **Loss of traceability:** there is no audit record of when an issue was resolved.

2.1 Technical Pieces vs. Pain Points

Technical piece	Pain point addressed	How it addresses it
Custom object (Support_Ticket__c)	No single entry channel	Centralizes every claim in one place. Priority is required so escalation logic cannot be bypassed; Closed Date enables auditing.
Trigger + Handler	No urgency differentiation	Automatically moves High-priority tickets to In Progress and stamps the closing timestamp when a ticket is closed.
Apex Controller	No backend visibility	Delivers open tickets ordered by real urgency (not alphabetically), always respecting the permissions of the logged-in user.
LWC (ticketDashboard)	No frontend visibility	Single panel where Lewis prioritizes and closes tickets in one click, with immediate visual feedback.
Permission Sets + Validation Rules	Uncontrolled changes and loss of traceability	Separate the reporting role from the management role, block priority downgrades, and prevent reopening closed tickets.

3. Architecture Justification

3.1 Why Not Escalation Rules (Case vs. Custom Object)

Salesforce native Escalation Rules operate only on the standard Case object. Because the incident data lives in a custom object (Support_Ticket__c) — chosen to model exactly the fields this process needs, without the overhead of Case — Escalation Rules cannot be applied to it. Programmatic logic is therefore required.

3.2 Why Apex and Not Flow

A before-save record-triggered Flow could implement the auto-escalation on its own, and it would be a legitimate choice for a production org with no other constraints. Apex was selected for three reasons:

- **Unit-tested guarantees:** Apex lets the behavior be verified by automated tests, including a 200-record bulk load and coverage measurement — the standard for enterprise-quality code.

- **One cohesive programmatic layer:** the Controller requires custom ordering by real urgency and enforcement of the user's permissions (WITH USER_MODE). Keeping the trigger logic in the same language and under the same test suite avoids splitting behavior between declarative and programmatic tools.
- **Trigger + Handler pattern:** the trigger contains no logic and delegates to a handler class, keeping responsibilities separated and the code reusable and testable.

3.3 Rubric Compliance

The chosen architecture directly supports the rubric criteria: a clear data model, a clean Trigger/Handler/Controller separation (Apex quality), a reactive LWC, bulk-validated tests above the coverage target, and documentation of every decision.

4. Data Model

The custom object **Support_Ticket__c** structures operational support requests. In addition to the standard **Name** field (configured as an Auto-Number with format ST-{0000}), the model incorporates six custom fields

Field	Type	Required	Purpose
Subject__c	Text (255)	Yes	Short title of the incident.
Description__c	Long Text Area	No	Detailed description of the problem.
Priority__c	Picklist: Low / Medium / High (default Low)	Yes	Drives auto-escalation and the sorting of the dashboard.
Status__c	Picklist: New / In Progress / Closed (default New)	No	Tracks the lifecycle stage of the ticket.
Contact__c	Lookup (Contact)	No	Identifies the person affected or requesting the support.
Closed_Date__c	Date/Time	No (system)	Stamped automatically when the ticket is closed.

4.1 Fields Added Beyond the Minimum

The four base fields (Subject, Description, Priority, Status) are enough to record and prioritize a ticket. Two fields were added deliberately to cover the traceability pain point:

- **Contact__c:** the system user who creates the ticket (Created By) is not necessarily the person affected. The lookup records who the incident is about, allows reporting per requester, and is optional so that urgent reports are never blocked by missing data.
- **Closed_Date__c:** without a closing timestamp it is impossible to measure resolution time or audit when a ticket was resolved. It is stamped by the Trigger and is read-only for the ticket creator, so it cannot be manually altered.

5. Implemented Business Logic

5.1 Trigger + Handler

The trigger only delegates to the handler class. The handler applies two business rules to every record in the batch (bulk-safe, no queries or DML inside loops):

- **Auto-escalation:** a High-priority ticket that is still New automatically moves to In Progress, on insert and on update, so critical incidents never wait for manual triage.

- **Closing timestamp:** when a ticket is Closed and Closed_Date__c is empty, the current date and time are stamped in it, on insert and on update. This covers tickets that change to Closed and tickets created already Closed, and it creates the audit record of resolution**.**

5.2 Apex Controller

- **Retrieving open tickets:** returns every ticket that is not Closed, enforcing the permissions of the current user. Results are ordered by real urgency (High, Medium, Low) and, within the same priority, oldest first.
- **Closing a ticket:** changes the status of the selected ticket to Closed, running as the current user so that field and object permissions apply. The Trigger then stamps the date.
- **Error handling:** any failure (invalid Id, missing permissions) is converted into a controlled exception that the interface can show to the user.

Why the ordering is custom: an early version sorted by the picklist field in the query (ORDER BY Priority__c DESC), and the tickets came back Medium, Low, High, which is not business urgency. A database sort on a picklist does not know that High is more urgent than Low. The fix was an explicit ranking rule in the Controller (High, then Medium, then Low), with ties broken by creation date — the behavior verified by the getOpenTicketsOrdersByPriorityCorrectly test in Section 7.

5.3 Lightning Web Component (ticketDashboard)

- **Reactivity:** the table is bound to the controller through the wire service, so it shows the current open tickets as soon as it loads.
- **Closing action:** each row has a Close Ticket button that calls the controller and then refreshes the data, so the ticket disappears from the list without reloading the page.
- **Visual feedback:** Success and Error toast notifications keep the operator informed about the result of every action.

5.4 Validation Rules

- **Prevent priority downgrade:** once a priority is assigned it cannot be lowered. Without this rule the ticket creator, who can edit Priority, could lower an escalated ticket and hide its urgency.
- **Prevent reopening closed tickets:** once a ticket is Closed, its status cannot change. This keeps the closing timestamp consistent and the audit trail trustworthy.

The formulas are included in Appendix A.

6. Security Model and Permissions

6.1 Permission Sets

	Support_Ticket_Creator (Angela James)	Support_Ticket_Manager (Lewis Webb)
Object	Read, Create, Edit. No Delete.	Read, Create, Edit, Delete.
Editable fields	Subject, Description, Priority, Contact.	All fields.except Closed Date and Name
Read-only fields	Status, Closed Date, Name.	Closed Date, Name
Apex access	Not required (does not use the dashboard).	Explicit access to TicketController.
Rationale	Can report and classify an incident but cannot change its lifecycle or overwrite audit data.	Manages the full lifecycle through the dashboard.

Note on Read-Only Fields:

- Name: Configured as an Auto-Number (ST-{0000}) and set to Read-Only across all Permission Sets to maintain sequence integrity.
- Closed Date: System-managed timestamp populated exclusively via Apex logic upon resolution.

For Salesforce Org setup screenshots, see Appendix A.5 & A.6 — Permission Sets Configuration**.**

6.2 Why Not a Permission Set Group

A Permission Set Group combines several permission sets into one assignable unit, which is useful when the same user needs several of them. Here each user has exactly one role and the roles do not overlap: Angela only reports, Lewis only manages, and the Manager set already includes everything the Creator has. A group would simply replicate the Manager set and add an administration layer without any benefit, so each user receives a single, easy-to-audit Permission Set.

6.3 Validation Rules as a Security Control

The Creator set must allow editing Priority so that Angela can classify incidents when she creates them. The priority-downgrade rule closes the gap this opens: it prevents an escalated ticket from being lowered afterwards, whoever edits it. The reopen rule protects closed tickets the same way. Both are declarative, so they apply regardless of how the record is edited (record page, dashboard, API, or data load).

Why the formula looks the way it does: *the first version of both rules compared the field's prior value directly to text with "=" — a picklist field cannot be reliably compared this way in a formula, and the rule silently failed to block anything. The fix was nesting PRIORVALUE inside ISPICKVAL, the function built specifically to compare a picklist's value against text (see* Appendix *A.2 and A.3).*

6.4 Field History Tracking

Field History Tracking is enabled on Support_Ticket__c for five fields: Status, Priority, Subject, Description, and Closed Date. Contact, Owner, and the Support Ticket Number are left untracked. Status and Priority are tracked because they are the two fields governed by the Validation Rules in Section 6.3 — those rules decide whether a change is *allowed*, while Field History records, for every change that *was* allowed, who made it and when. Subject and Description are tracked because they are the fields the Creator can still edit after submitting a ticket, so any correction to the original report stays visible. Closed_Date__c is tracked as the anchor of the audit trail itself. Together with the Permission Sets (6.1) and the Validation Rules (6.3), this

completes the security model: who can act, what they are blocked from doing, and a permanent record of what actually happened. *(See Appendix A.8 — Field History Tracking Configuration)*.

7. Quality Assurance and Testing Strategy

Nine automated test methods cover the Trigger, the Handler, and the Controller, including 200-record bulk operations to verify compliance with governor limits.

7.1 SupportTicketTriggerTest (4 methods)

Test	Intent
Single record escalation	Verifies that a High-priority ticket is automatically moved to In Progress.
Low priority control	Confirms that Low-priority tickets remain New, proving that escalation applies only when it should.
Full lifecycle and closure	Validates the update path: High-priority behavior and the generation of Closed_Date__c when the ticket is closed.
Bulk load validation	Inserts 200 High-priority tickets at once to confirm the logic is bulk-safe and does not hit governor limits.

7.2 TicketControllerTest (5 methods)

Test	Intent
Query filter validation	Confirms that only open tickets are returned and Closed ones are excluded.
Custom priority sorting	Verifies the High, Medium, Low order, proving that urgency is not sorted alphabetically.
Bulk retrieval	Validates retrieval of 200 records in a single call.
Imperative closure	Verifies that closing a ticket changes its status to Closed.
Exception handling	Tests error recovery when an invalid ticket Id is supplied: a controlled exception must be thrown.

7.3 Coverage Results

Class	Coverage	Lines
SupportTicketTrigger	100%	1 / 1
SupportTicketTriggerHandler	100%	13 / 13
TicketController	89%	25 / 28
Overall	92%	39 / 42

The overall figure is 92% against a platform minimum of 75%. The three uncovered lines of the Controller are defensive paths not reached in normal operation: the exception handling of the ticket query and the default branch of the priority ranking, which would only apply to a value outside the picklist. They were not forced artificially, because the critical business paths are fully covered.

Note: For the Developer Console test execution screenshot showing 9/9 passed test methods and full code coverage breakdown (92%), see **Appendix** A.7 Apex Test Execution & Code Coverage Metrics

7.4 Automatic Rollback and Test Isolation

Each test creates its own data, and Salesforce runs it inside a transaction that is automatically rolled back when the test ends. Nothing created by a test persists in the org, so tests are repeatable, independent of each other and of existing org data. This is what makes it safe to insert 200 records per test and run the suite as many times as needed.

7.5 A Development Lesson: Test Data Must Track the Data Model

Required-field error during bulk testing: once Subject__c was set as required at the object level, the earliest test methods — written before that change — began failing with REQUIRED_FIELD_MISSING, because they built ticket records without a Subject value. Every test record, across both test classes, now sets Subject__c explicitly. This confirmed that the object's own validations and the test code are coupled: a configuration change can break tests that used to pass, with no Apex line touched.

Support Ticket Number not rendering on the dashboard: early in development, the Support Ticket Number (Name) column appeared blank on the ticketDashboard table, even though every ticket had a valid value. Root cause: the Controller's query explicitly lists only the fields the business logic needs — unlike Id, Name is not included automatically unless it is selected. Fix: Name was added explicitly to the query's field list.

8. Deployment, Tooling, and SFDX Integration

- **Development environment:** Visual Studio Code with the Salesforce Extension Pack.
- **Test execution**: automated unit tests run with the SFDX CLI (command in Appendix A) and were verified in the Developer Console (screenshots in Appendix A.7).
- **Version control and structure**: standardized SFDX directory structure (force-app/main/default) for source-driven deployment and org synchronization.
- **Source code repository**: the project, including the data model (custom object, fields, Validation Rules and Permission Sets retrieved with the Salesforce CLI), is published on GitHub:

  https://github.com/marisaeli11/support-ticket-tracker-salesforce

9. Manual Test Guide — Full Lifecycle of Ticket ST-0010

To keep the story easy to follow, the business behavior was validated end-to-end on a single ticket, ST-0010, walked through its complete lifecycle rather than spreading the demonstration across unrelated tickets. Technical mechanics already covered in Sections 5 and 6 are not repeated here. The full set of screenshots for every step is in Appendix B; the two most representative moments are shown below.

#	Step	Who	What it demonstrates
1	Creation — ST-0010, Priority = Medium	Angela	Status stays in New, the field's native default.
2	Downgrade attempt (blocked)	Angela	Prevent_Priority_Downgrade rejects the attempt — priority can move forward, never backward.

3	Escalation — Priority changed to High.	Angela	Status moves to In Progress automatically, with no manual action.
4	Ticket seen in the work queue	Lewis	ST-0010 appears grouped with the other High tickets, ordered oldest-first.
5	Closure	Lewis	The ticket leaves the dashboard; the record shows Closed_Date__c stamped automatically.
6	Reopen attempt (blocked)	Lewis	Prevent_Reopen_Closed_Ticket rejects the attempt, keeping the closing date permanently trustworthy.

Note: *For the complete step-by-step screenshot sequence of the ST-0010 lifecycle (Steps 1–6), see Appendix B — Manual Test Evidence (ST-0010 Full Lifecycle)*

10. Improvement Opportunities

These are known limits of the current version. They are documented because understanding what the solution does not yet cover is part of the engineering work.

10.1 Closed Date on Creation and in Data Imports

- Current behavior: the Handler stamps Closed_Date__c on insert and on update when the status is Closed and the date is empty. Creating a ticket directly as Closed was verified manually (ST-0013). Because the Handler only stamps an empty date, a historical date that comes with an import is kept. Neither case is covered by an automated test.
- Why it matters: tickets created already closed (by Lewis, an administrator, or a data load of historical tickets) must keep a correct Closed Date to protect the audit trail and the resolution-time metrics.
- Proposal: add automated tests that (1) insert a ticket as Closed with an empty date and assert it is stamped, and (2) insert tickets with a historical Closed_Date__c and assert it is not overwritten. Then validate it with a real import.

10.2 Security Tests with System.runAs

- Current behavior: tests run as the administrator who executes them, who has access to everything, so WITH USER_MODE never rejects anything in the tests. The tests prove the logic but not the security model.
- Why it matters: the separation of roles between Angela and Lewis is a core requirement, and it has only been verified manually.
- Proposal: run tests as users with each Permission Set: the Manager can query and close, the Creator cannot close, and a user without permissions receives a controlled error.

10.3 Email Notification for the Ticket Manager

- Current behavior: Lewis has to open the dashboard to find out that a new High priority ticket exists.
- Proposal: send an email to the ticket manager when a High priority ticket is created. It needs an after trigger (or an asynchronous process), because the ticket must exist first. It must stay bulk-safe: one email message for the whole list of tickets, never one per record inside a loop.

10.4 Filter the Dashboard by Priority and Status

- Proposal: let the manager focus only on the urgent tickets. It can be done in the LWC with the data the Controller already returns, without changing the Apex code.

10.5 Summary of Open Tickets

- Proposal: show how many tickets are open by priority and status, using a grouped query in the Controller or a standard Salesforce Dashboard.

10.6 Jest Tests for the LWC

- Current behavior: the dashboard is validated manually and through the Apex tests.
- Proposal: add Jest tests that mock the wire adapter and the Close Ticket action.

10.7 Alternative Evaluated: Priority_Rank__c Formula Field

- A formula field converting priority to a number (High 1, Medium 2, Low 3) would allow ordering directly in the query and remove the comparison logic from Apex. It was evaluated and not adopted, because it adds a seventh field, requires extra field permissions in both Permission Sets, and the custom comparator is already tested. It remains a valid option if the sorting needs to be reused in reports.

Appendix A — Technical Reference

A.1 Controller Method Signatures

```
@AuraEnabled(cacheable=true)
public static List<Support_Ticket__c> getOpenTickets() { ... }

@AuraEnabled
public static void closeTicket(Id ticketId) { ... }
```

A.2 Validation Rule: Prevent Priority Downgrade

```
AND(
  ISCHANGED(Priority__c),
  OR(
    AND(
      ISPICKVAL(PRIORVALUE(Priority__c), "High"),
      NOT(ISPICKVAL(Priority__c, "High"))
    ),
    AND(
      ISPICKVAL(PRIORVALUE(Priority__c), "Medium"),
      ISPICKVAL(Priority__c, "Low")
    )
  )
)
```

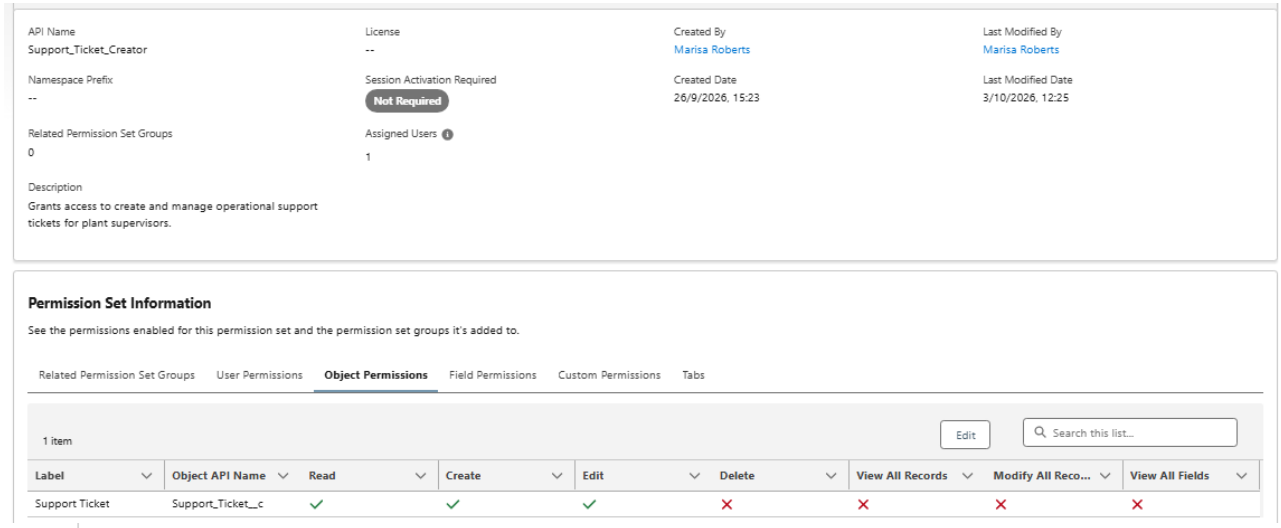
A.3 Validation Rule: Prevent Reopening a Closed Ticket

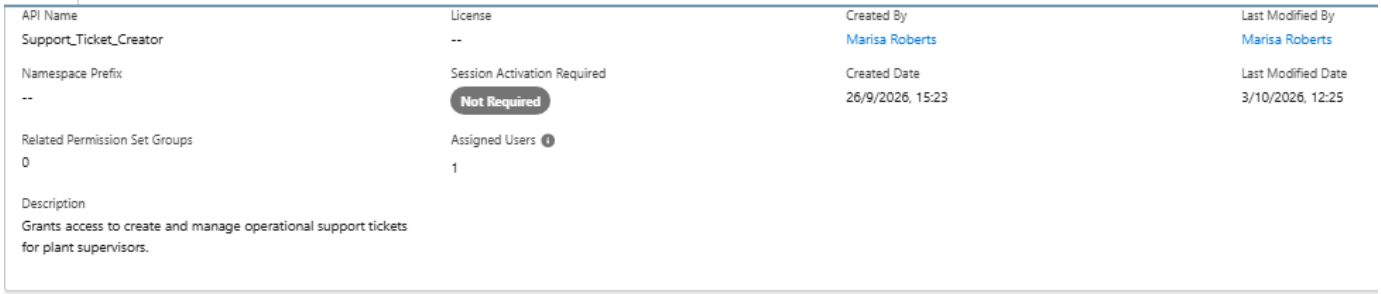
```
AND(
  ISCHANGED(Status__c),
  ISPICKVAL(PRIORVALUE(Status__c), "Closed")
)
```

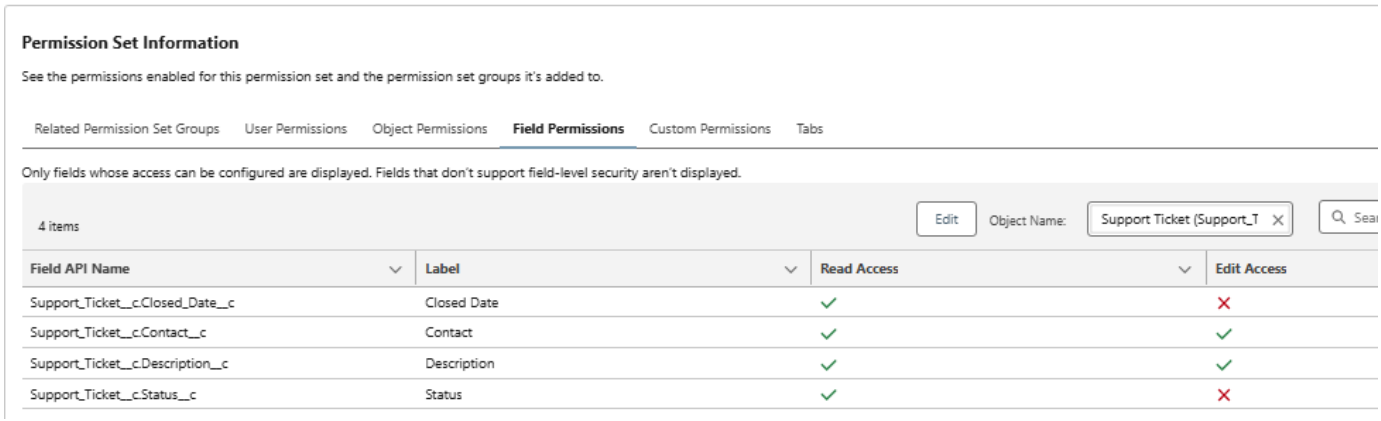
A.4 Test Execution Command

```
sf apex run test --test-level RunLocalTests --result-format human --code-coverage
```

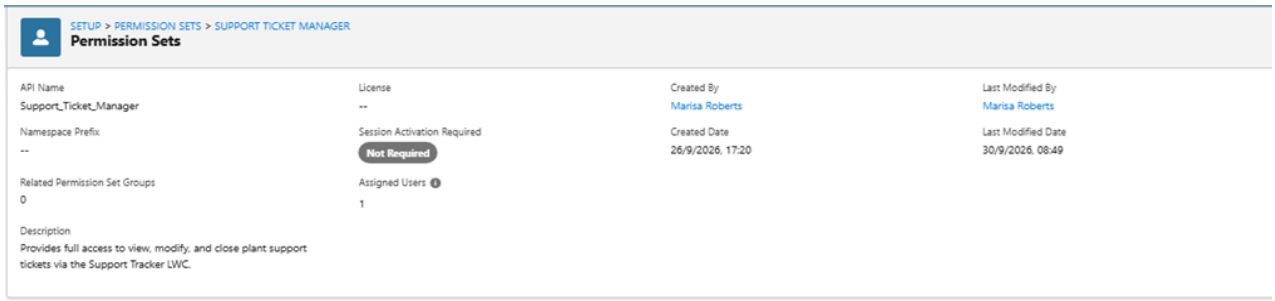
A.5 Permission Set Support Ticket Creator

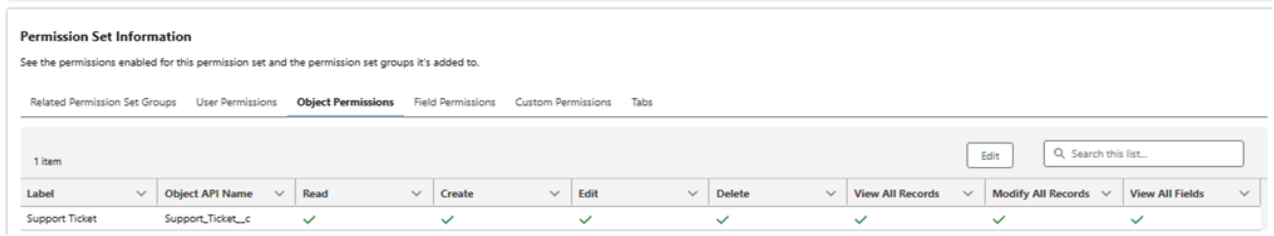


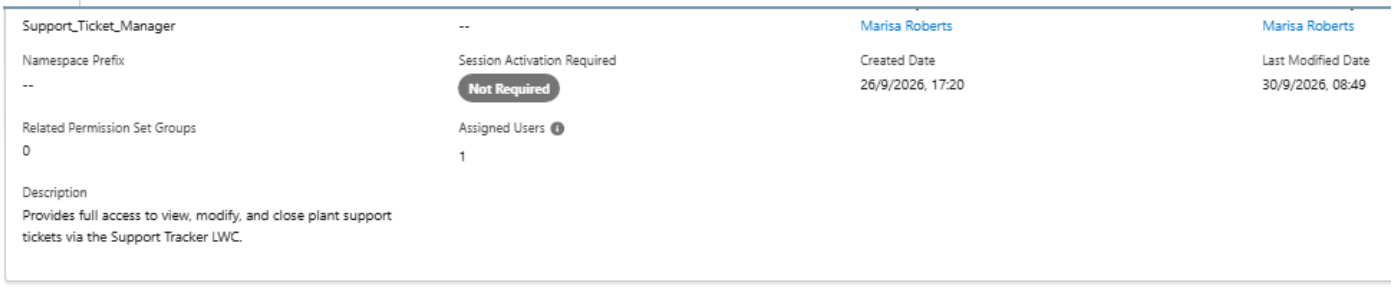


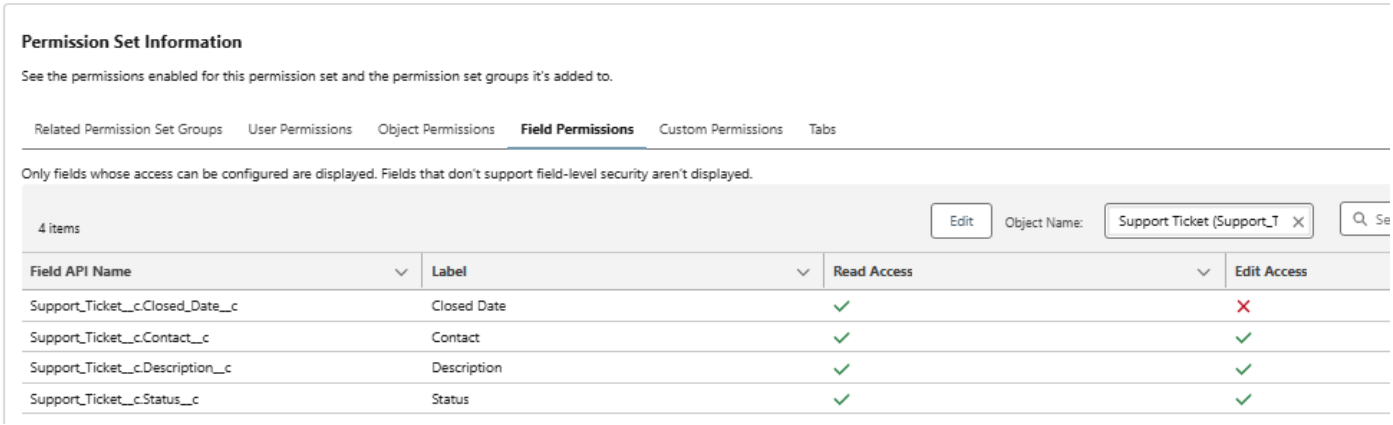


A.6 Permission Set Support Ticket Manager

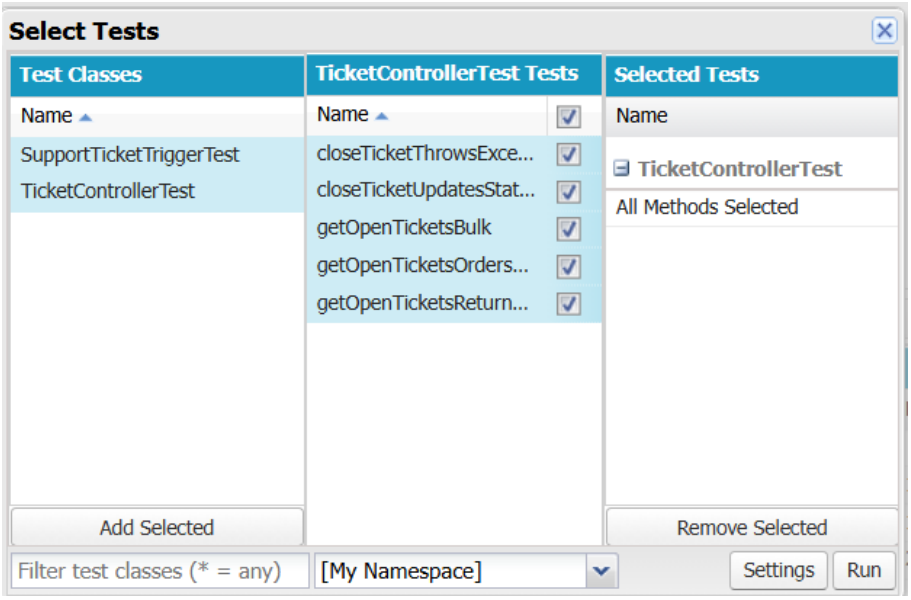


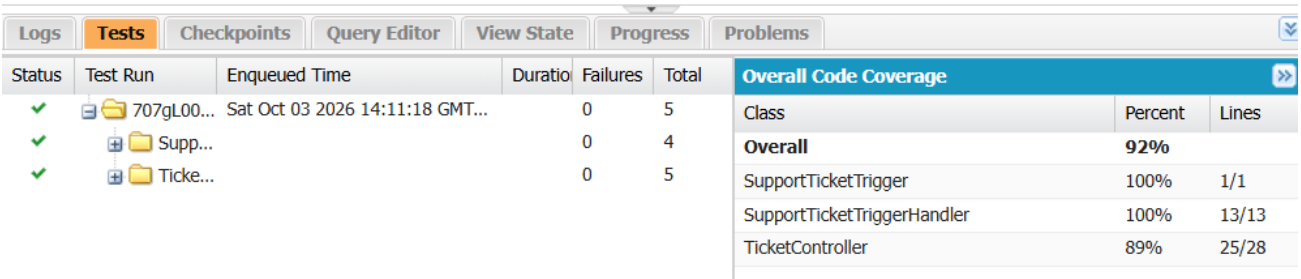




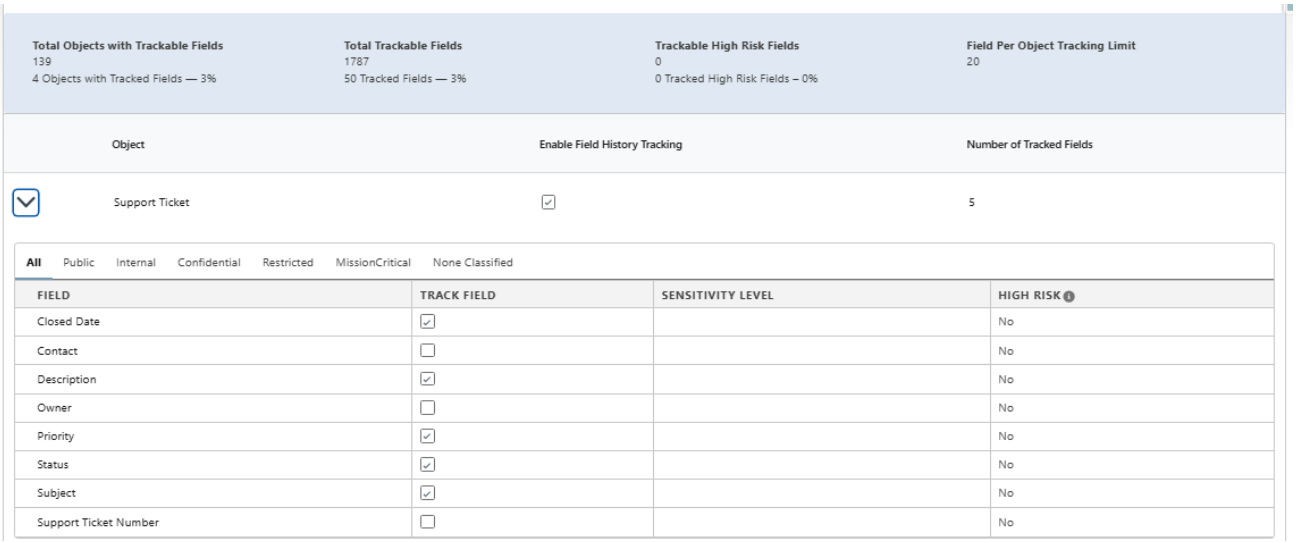


A.7 Apex Test Execution & Code Coverage Metrics





A.8 — Field History Tracking Configuration)



Total Objects with Trackable Fields
139
4 Objects with Tracked Fields — 3%

Total Trackable Fields
1787
50 Tracked Fields — 3%

Trackable High Risk Fields
0
0 Tracked High Risk Fields – 0%

Field Per Object Tracking Limit
20

Object	Enable Field History Tracking	Number of Tracked Fields
Support Ticket	☑	5

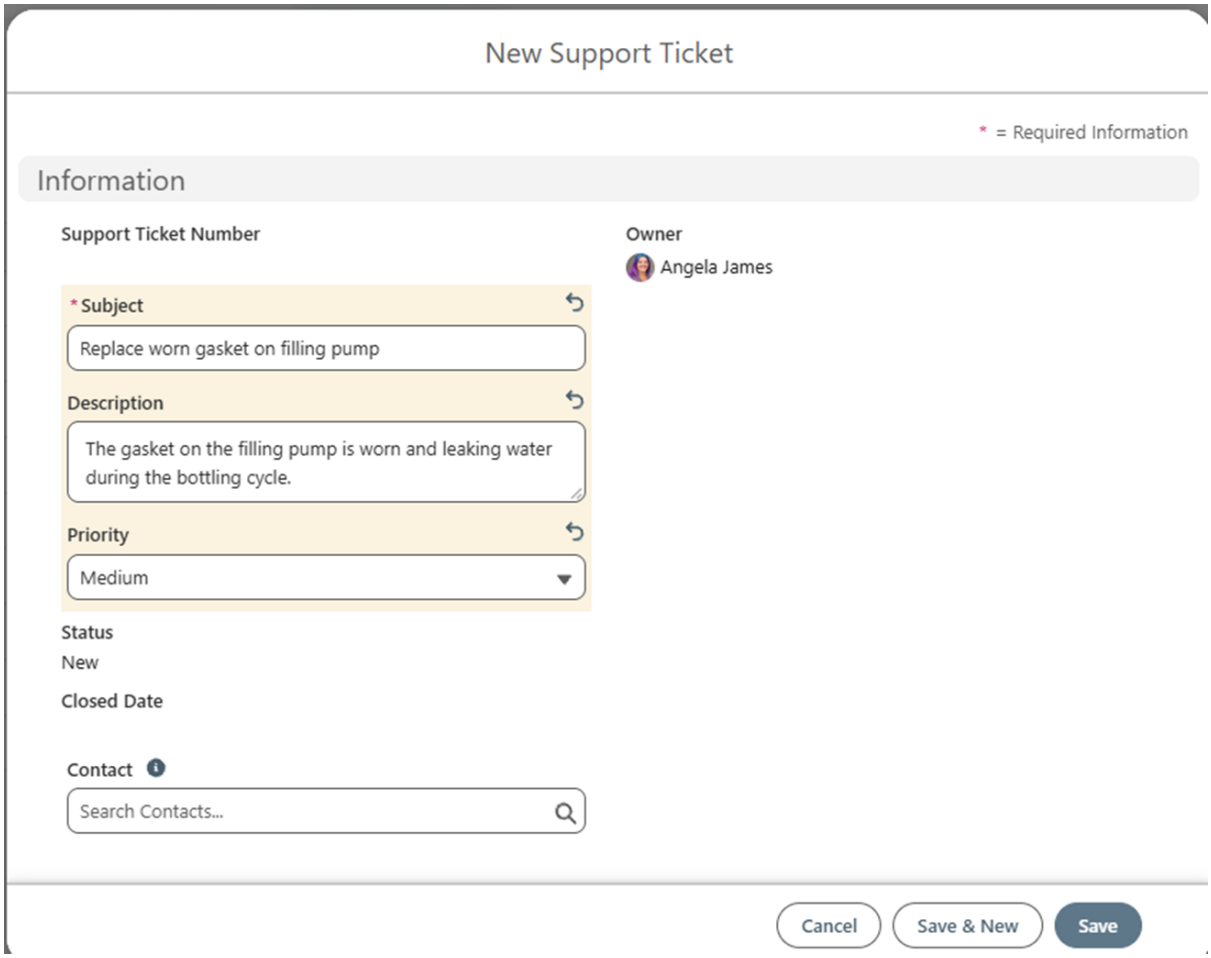
All | Public | Internal | Confidential | Restricted | MissionCritical | None Classified

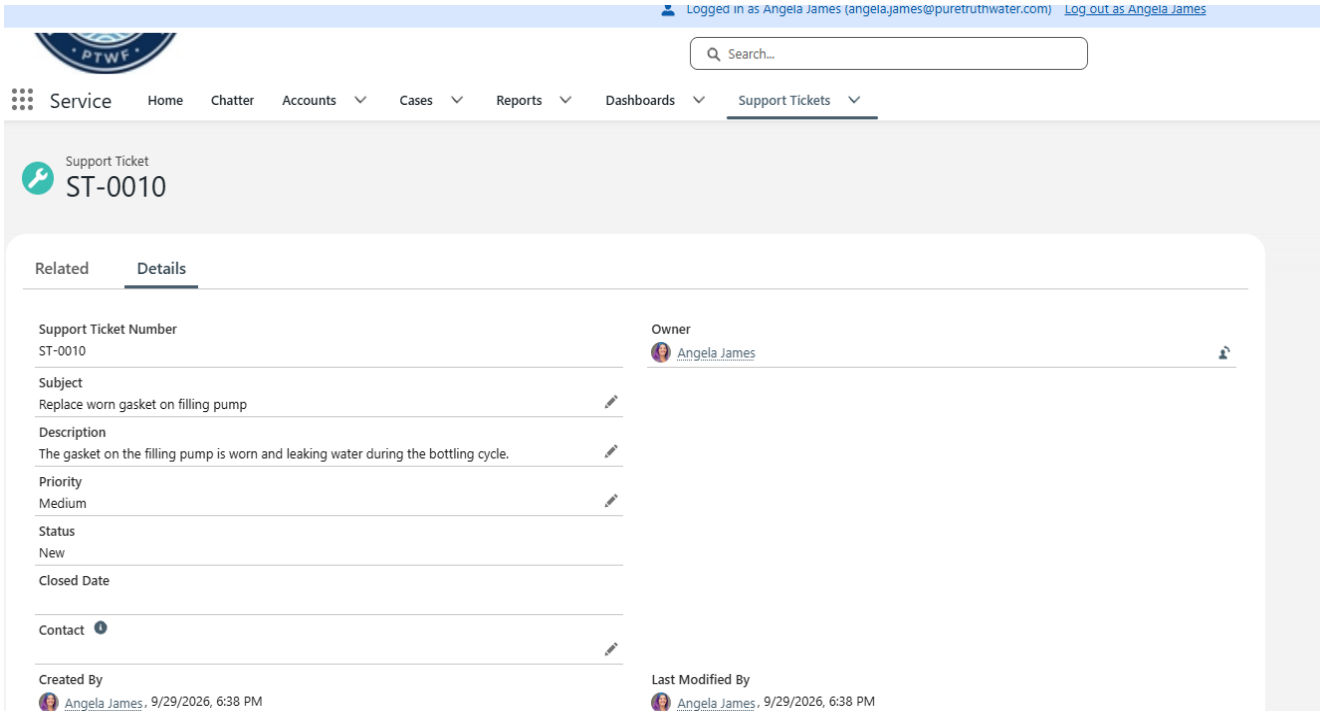
FIELD	TRACK FIELD	SENSITIVITY LEVEL	HIGH RISK
Closed Date	☑		No
Contact	☐		No
Description	☑		No
Owner	☐		No
Priority	☑		No
Status	☑		No
Subject	☑		No
Support Ticket Number	☐		No

Appendix B — Manual Test Evidence (ST-0010 Full Lifecycle)

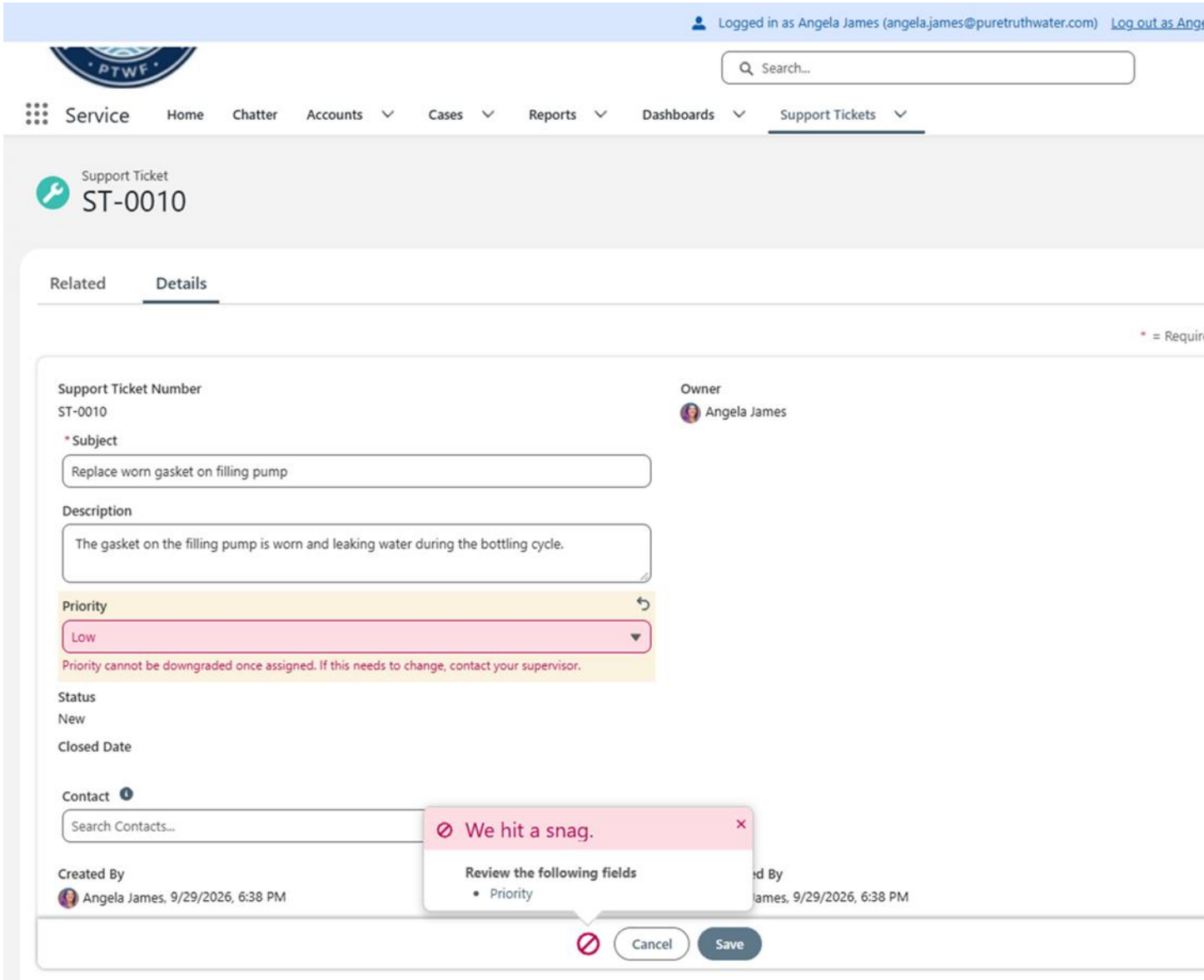
Full screenshot sequence supporting the six steps summarized in Section 9.

Step 1 — Creation, ST-0010, Priority = Medium, Status = New

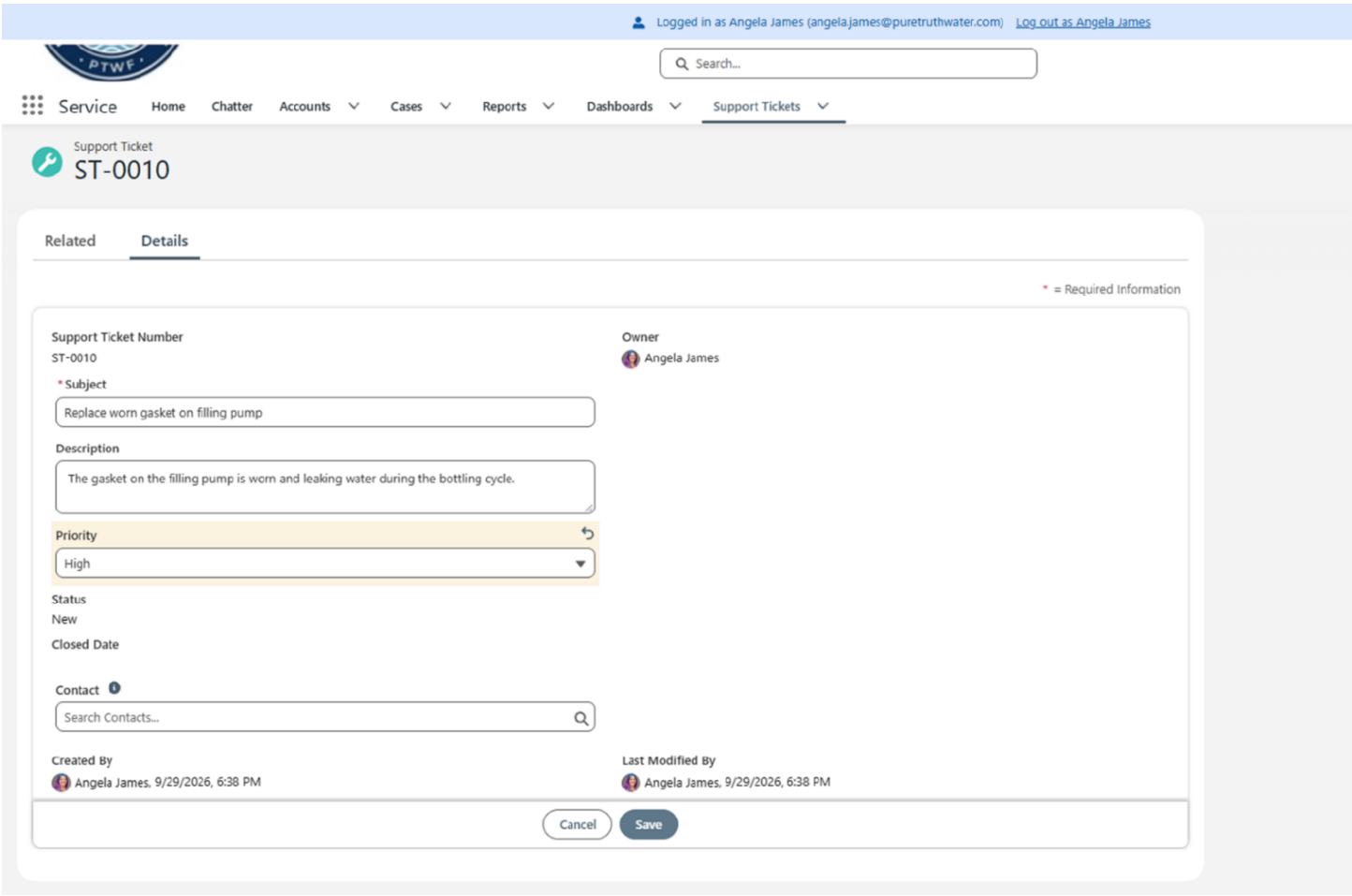




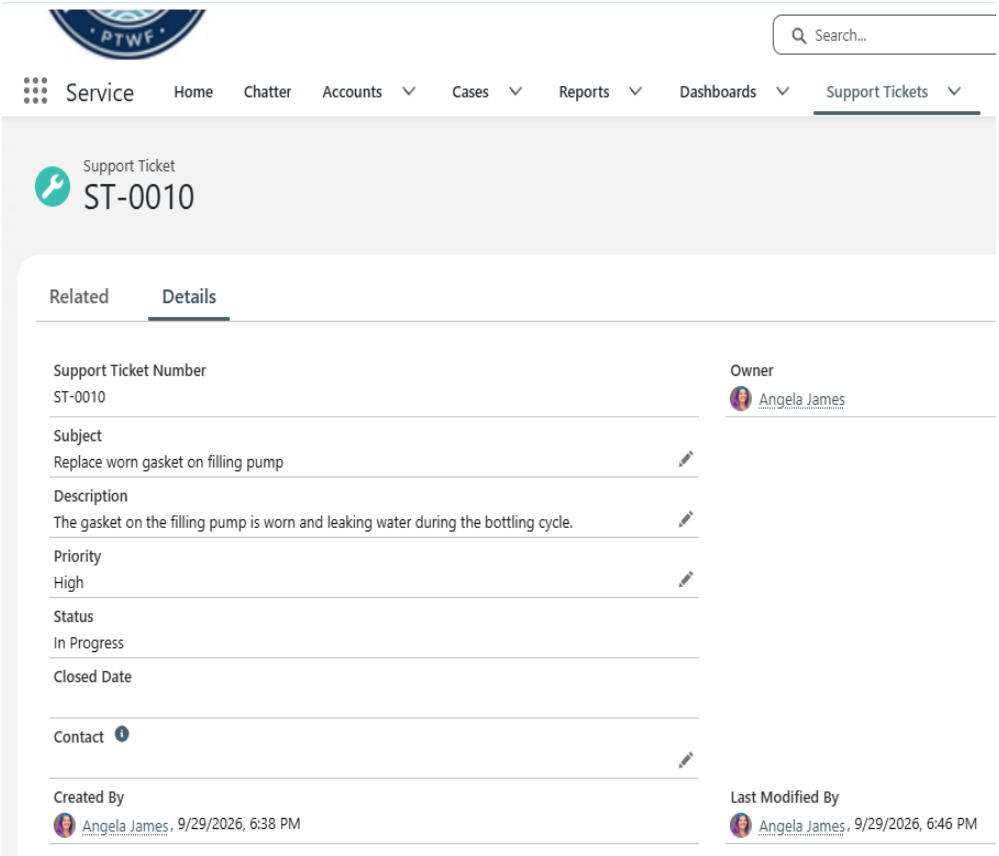
Step 2 — Downgrade attempt blocked (Validation Rule error)



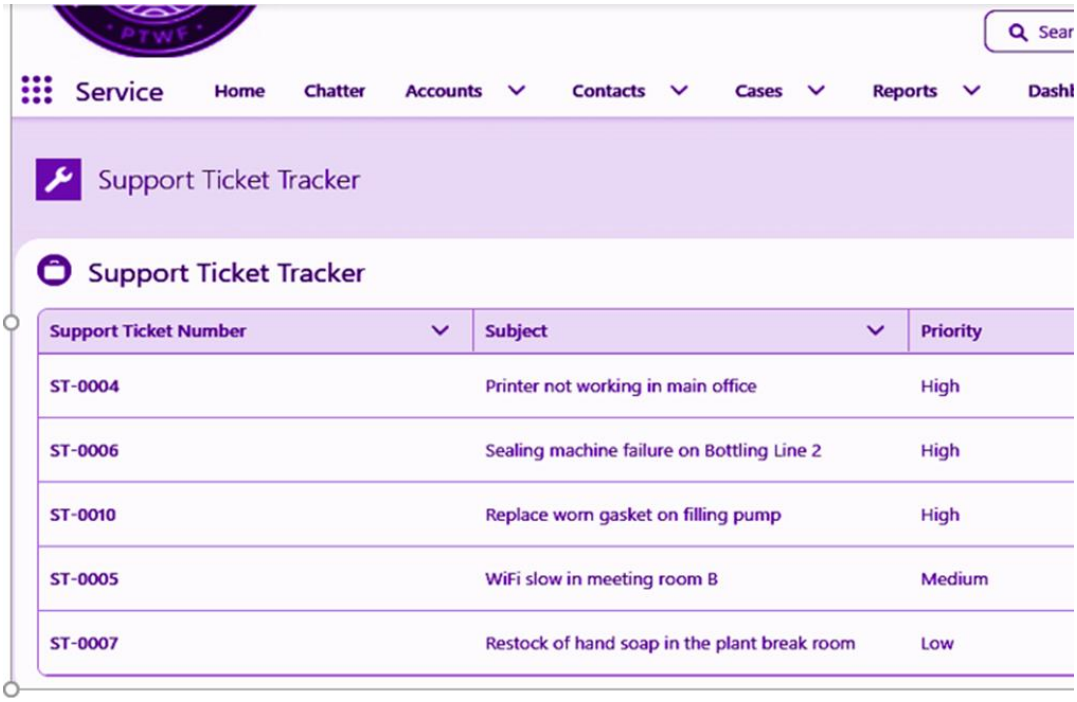
Step 3 — Escalation, Priority = High, Status = In Progress



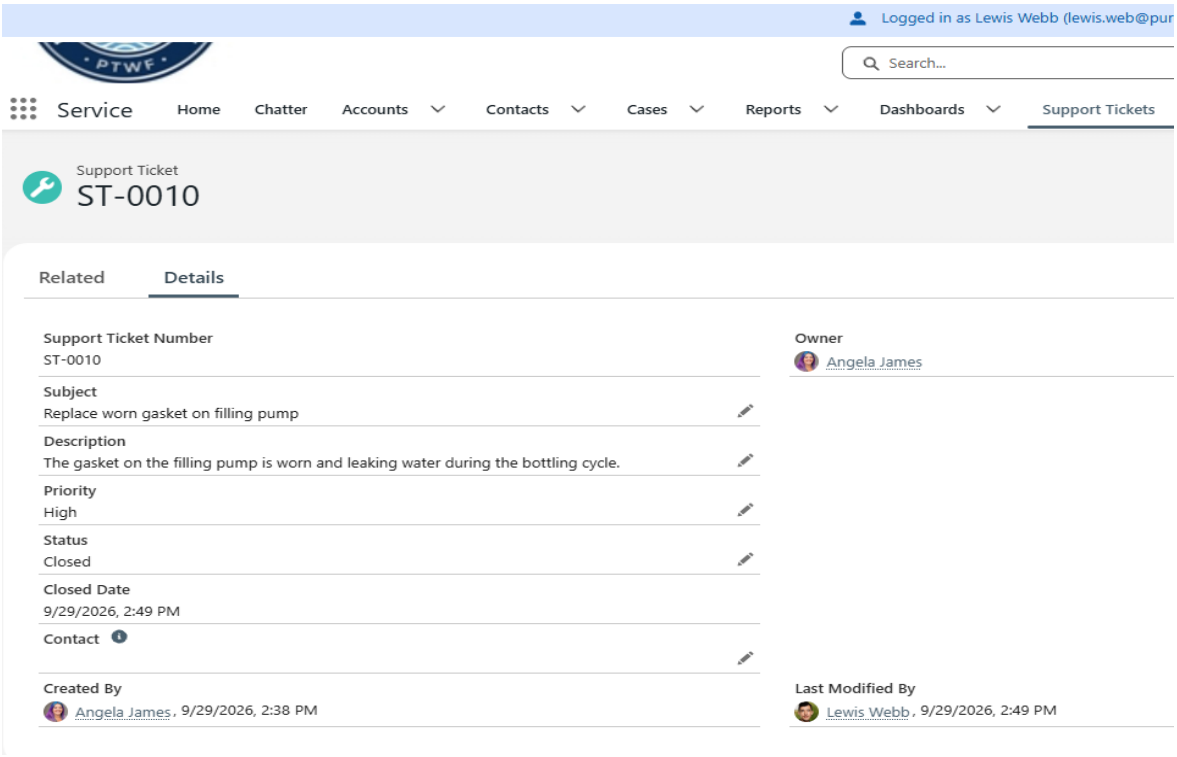
After SAVE Status: In progress

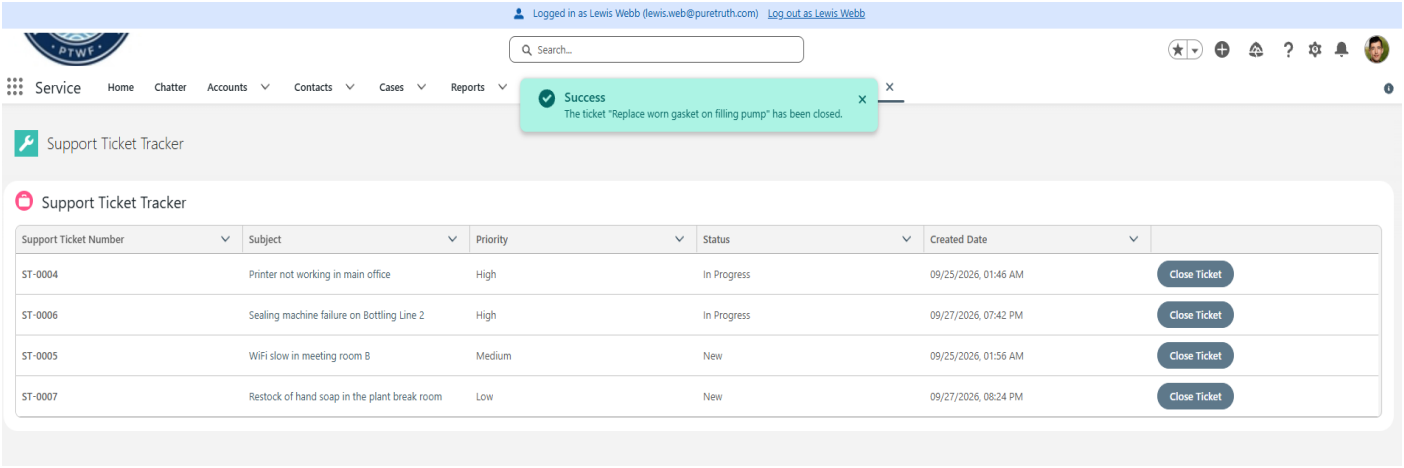


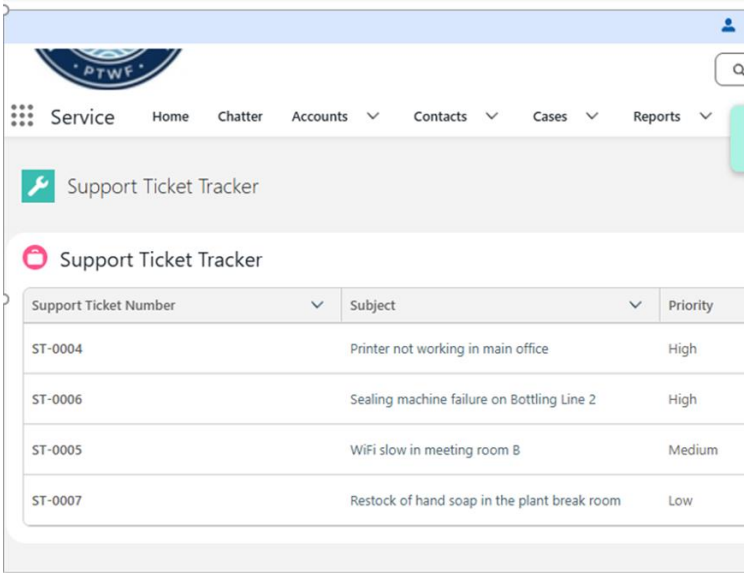
Step 4 — Ticket in Lewis's work queue, grouped with other High tickets



Step 5 — Closure, Closed_Date__c stamped automatically







Step 6 — Reopen attempt blocked (Validation Rule error)

